Trainable Resonant Neurons: End-to-End Differentiable Envelope Extractors with Analog Filters

Hariz Zoran Farooq[1,*]

[1]Independent Researcher [*]Corresponding author:
harizzoranfarooq@gmail.com

***Abstract*—This paper introduces Trainable Resonant Neurons (TRNs), a differentiable component for Analog Neural Networks (ANNs) that introduces adaptive spectral filtering. Each TRN combines a bandpass filter with trainable center frequency (f_0) and quality factor (Q), followed by an envelope detector, enabling end-to-end learning of frequency-selective features from raw signals. The architecture is benchmarked against a Fixed Filter Bank (FFB) on two classification tasks. While both perform similarly on the simpler task, the gap widens under higher spectral and noise complexity: the FFB peaked at 78.45% Accuracy, whereas the TRN reached 84.69%. Although this margin may seem modest, it represents a lower bound under controlled conditions; the advantage is expected to increase substantially for real-world, high-complexity data.**

I. Introduction

This paper introduces the design of a Trainable Resonant Neuron (TRN), a novel alternative to conventional filter banks in Analog Neural Networks (ANNs) for signal processing and pattern identification. Circuit implementations are considered under ideal components. Unlike traditional filter banks that rely on multiple fixed filters across the spectrum, a TRN directly trains both the center frequency and quality factor, offering fine control of filtering characteristics while reducing the number of required filters, maintaining selectivity and precision. TRNs thus have the potential to significantly decrease ANN size and complexity while enhancing efficiency, adaptability, and overall performance. In this sense, they embody the same adaptability that has replaced handcrafted features with learnable filters in deep learning, but in a fully circuit-realizable and analytically tractable form.

Although initially introduced as an input-stage neuron for frequency-selective envelope extraction, the TRN architecture is flexible enough to be reconfigured for other functions. By omitting rectification and envelope detection, it can operate entirely in the AC domain, enabling additional modes of functionality beyond its original scope. This flexibility makes the TRN not only an efficient replacement for filter banks but also a versatile building block for broader classes of analog and hybrid neural networks, with potential applications in adaptive sensing, neuromorphic computation, intelligent front-end processing, and beyond.

II. Literature Review

Early work on adaptive filtering, such as the Adaptive Learning Network (ALN), addressed cases where the ideal filter was not known in advance. ALNs learned energy estimation from empirical data using transversal Finite Impulse Response (FIR) structures, ensuring stability but functioning in a black-box manner [1]. In contrast, approaches in computer vision and audio rely on learnable filter layers embedded in Convolution Neural Networks (CNNs). For instance, Learnable Bandpass Filters (LBFs) were introduced to suppress moiré patterns by replacing handcrafted kernels with differentiable filters [2], while Learnable Filter (LF) layers parameterize audio bandpass responses with functions like Gammatone or Sinc, reducing parameter count and aligning with psychoacoustic scales [3].

More recently, parametric filter banks have been developed to provide interpretable and tunable front-ends for audio applications, further emphasizing structured, learnable frequency-selective representations [4]. Similarly, SincNet directly constrains CNN front end filters to sinc-shaped bandpasses, learning only cutoff frequencies [5]. In all these cases, convolutional layers become interpretable and efficient but remain purely digital and domain specific.

Other methods, such as COPE (Combination of Peaks of Energy), learn representations of environmental sounds by configuring feature extractors from prototype patterns. COPE emphasizes robustness to low signal-to-noise ratios and models auditory energy peaks but remains non-parametric and application specific [6]. Likewise, EEGminer introduces generalized Gaussian filters for EEG analysis, emphasizing smooth differentiability and interpretability of brain rhythms [7]. Both methods highlight noise robustness but rely on software pipelines and lack physically interpretable circuit parameters.

Recent work has also sought to embed analog filter structures into differentiable systems. Neural Analog Filters (NAFs) approximate convolutional weights with latent analog filter representations, improving robustness to sampling-frequency mismatch in source separation tasks [8]. Similarly, AI-driven analog circuit co-design has demonstrated joint optimization of analog front-end parameters with classifier performance, incorporating hardware constraints into the loss [9]. These approaches directly integrate analog models into learning loops but focus on domain specific objectives and require complex optimization schemes.

Collectively, these works illustrate the trend of making fixed filter banks with adaptive alternatives. Yet none fully equates to the Trainable Resonant Neuron (TRN). ALNs lack parametric interpretability, CNN-based filters remain digital and domain restricted, prototype-driven methods like COPE are non-parametric, and analog co-design focuses on hardware optimization rather than generalizable neuron architecture. TRNs uniquely integrate a parametric resonant filter with explicit trainability (f_0, Q), intrinsic nonlinearity (rectification and envelope smoothing), and a formal responsivity–stability tradeoff (ρ, κ). This combination makes them simultaneously circuit realizable, differentiable, interpretable, and general-purpose, positioning TRNs as a distinct contribution beyond existing approaches.

III. Overview

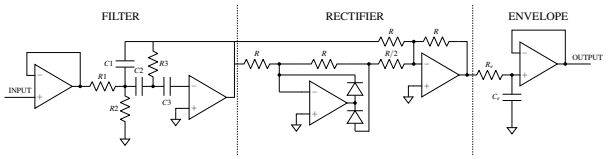

Figure 1: Full-wave rectifier-based envelope extraction circuit.

System Stage	Description
Filter Stage	An input signal passes through a buffer to a bandpass Multiple Feedback (MFB) filter which extracts a specific frequency range from the input signal using resistors R_1, R_2, and R_3; and capacitors C_1, C_2, and C_3.
Rectifier Stage	Full-wave rectification converts the AC output from the filter stage into a unipolar signal using resistors R and $R/2$.
Envelope Stage	RC smoothing filter generates amplitude envelopes from the rectified signals using resistor R_e and capacitor C_e, outputting it through a buffer.

Table 1: Overview of the signal processing stages in the system.

IV. Filter Stage

The neuron design employs a Multiple Feedback (MFB) filter, enabling adjustment of both the center frequency and the quality factor, without the need for inductive components. Nevertheless, instability due to resonance remains an unavoidable concern, as it is inherently linked to the quality factor Q.

To ensure consistency across all input neurons, the system must maintain unity gain at the reference frequency ω_0, a requirement that is also supported by the MFB filter's gain control. All of these functions are realized within a single op-amp, thereby enhancing practicality.

$$H_{BP}(s) = \frac{\frac{\omega_0}{Q} K s}{s^2 + \frac{\omega_0}{Q} s + \omega_0^2} \tag{1}$$

Substituting $s = j\omega$ and setting the gain K at ω_0 as 1 gives the magnitude:

$$|H_{BP}(j\omega)| = \frac{\frac{\omega_0}{Q}\omega}{\sqrt{(\omega_0^2 - \omega^2)^2 + \left(\frac{\omega_0 \omega}{Q}\right)^2}} \tag{2}$$

Setting Q for this second-order filter is straightforward; however, this can be formed into a higher Nth-order bandpass filter:

$$|H_{BP}(j\omega)|^N = \left(\frac{\frac{\omega_0}{Q}\omega}{\sqrt{(\omega_0^2 - \omega^2)^2 + \left(\frac{\omega_0 \omega}{Q}\right)^2}} \right)^N \tag{3}$$

The $-3\,\mathrm{dB}$ points satisfy:

$$|H_{BP}(j\omega_{-3\mathrm{dB}})|^N = \frac{1}{\sqrt{2}} |H_{\mathrm{BP}}(\omega_0)|^N \tag{4}$$

This leads to an exact formula for Q for the Nth-order filter.

$$Q_{\mathrm{eff}}^{(N)} = \frac{Q}{\sqrt{2^{1/N} - 1}} \tag{5}$$

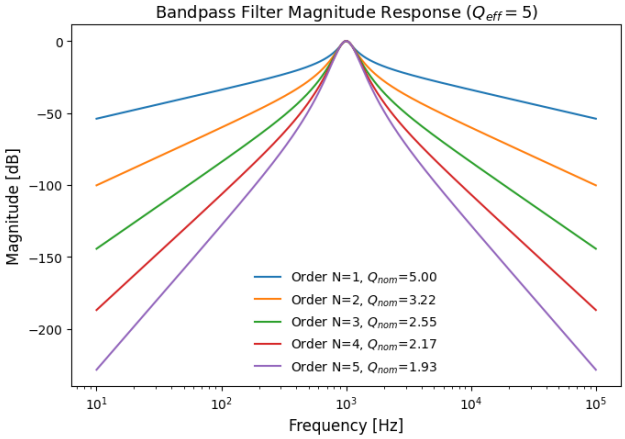

Figure 2: Magnitude response comparison of cascaded filters.

The instability due to resonance is also magnified by increasing the order.

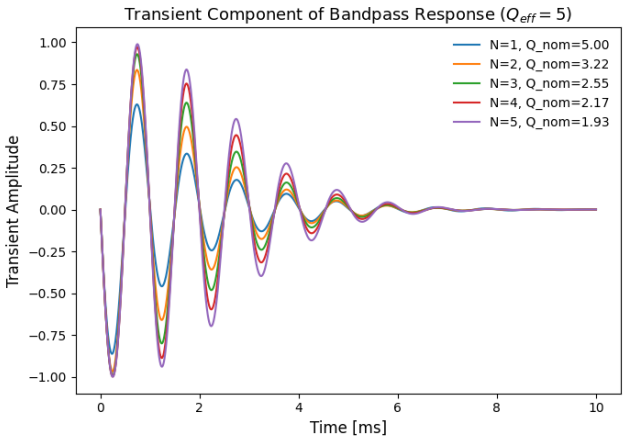

Figure 3: Transient response comparison of cascaded filters.

The instability is estimated using a simple method that avoids higher-order arithmetic: the output for a sinusoidal input at frequency ω_0 is subtracted from the input $\sin(\omega_0 t)$ to isolate the transient component.

For this simulation, the differential equation alternative is used, provided later in Equation 23.

V. RC Smoothing

The response to a constant-frequency, constant-amplitude sinusoidal input exhibits a voltage ripple that increases periodically until reaching a steady state, oscillating around a fixed mean. The time to reach steady state measures responsiveness, while the ripple amplitude reflects stability. These traits trade off: as one is stronger, the other becomes weaker.

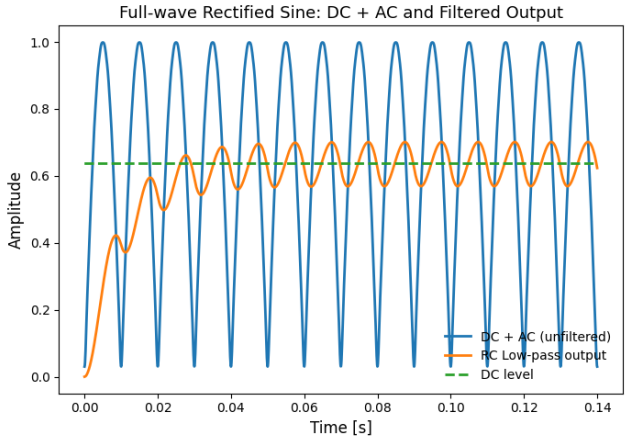

Figure 4: Breakdown of RC smoothing.

These metrics can be standardized by defining stability as the ratio of ripple amplitude to input amplitude, and responsiveness as the ratio of settling time (to within a specified error band) to the input signal period.

To evaluate these metrics, the input is taken as a constant-frequency, constant-amplitude sinusoidal waveform, which originates from the filter, rectified and then passed into the RC smoothing network at the end of the circuit:

$$A\left|\sin(\omega t)\right| \tag{6}$$

Using the Fourier series expansion of the input:

$$A\left|\sin(\omega t)\right| = \underbrace{\frac{2A}{\pi}}_{\text{DC term}} - \underbrace{\frac{4A}{\pi}\sum_{n=1}^{\infty}\frac{\cos(2n\omega t)}{4n^2-1}}_{\text{AC terms}} \tag{7}$$

This provides a breakdown of the input to be analyzed separately in the DC and AC domains, where the DC domain pertains to the underlying voltage that determines the steady state and, hence, the responsiveness, while the AC domain pertains to the oscillations superimposed on this voltage, which determine the stability.

$$V_c^{\mathrm{DC}}(t) = \frac{2A}{\pi}\left(1-e^{-t/RC}\right) \tag{8}$$

Set $V_c(t) = \frac{2A}{\pi}(1-\varepsilon)$, where ε is the tolerance (e.g. $\varepsilon = 0.01$ for 1%):

$$t = RC\cdot\ln\left(\frac{1}{\varepsilon}\right) \tag{9}$$

The ratio of this time t to the input signal's period $T = \frac{2\pi}{\omega}$ is:

$$\frac{t}{T} = \frac{RC\cdot\ln\left(\frac{1}{\varepsilon}\right)}{2\pi/\omega} = \frac{\omega RC}{2\pi}\ln\left(\frac{1}{\varepsilon}\right) \tag{10}$$

The magnitude of the transfer function of a simple RC low-pass filter is:

$$|H(j\omega)| = \frac{1}{\sqrt{1+(\omega RC)^2}} \tag{11}$$

Using this in the AC expression of the Fourier transform provides:

$$V_c^{\mathrm{AC}}(t) = -\frac{4A}{\pi}\sum_{n=1}^{\infty}\frac{\left|H(j2n\omega)\right|}{4n^2-1}\cos\Big(2n\omega t + \angle H(j2n\omega)\Big) \tag{12}$$

The normalized (relative measure) AC waveform for the stability metric is:

$$\frac{V_c^{\mathrm{AC}}(t)}{A} = -\frac{4}{\pi}\sum_{n=1}^{\infty}\frac{\left|H(j2n\omega)\right|}{4n^2-1}\cos\Big(2n\omega t + \angle H(j2n\omega)\Big) \tag{13}$$

For a first-order RC low-pass filter, any degree of smoothing requires the fundamental ripple component ($n=1$ at frequency 2ω) to lie beyond the cutoff frequency $f_c = \frac{1}{2\pi RC}$. Above cutoff, the filter attenuates at exactly -6 dB per octave, i.e. the amplitude is halved with each frequency doubling.

Hence, above the cutoff, the attenuation of each harmonic can be found using:

$$Att(n) = \frac{1}{2n}\frac{1}{4n^2-1} = \frac{1}{8n^3-2n} \tag{14}$$

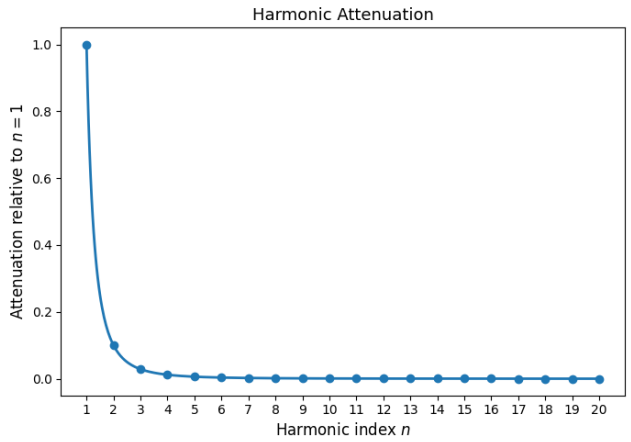

Figure 5: Comparison of attenuation of harmonics due to Fourier transform and magnitude response.

Thus, in the smoothing regime the $n = 1$ term dominates the ripple, and higher-order harmonics may be neglected.

$$V_{\text{ripple}}(t) = -\frac{4}{3\pi}\left|H(j2\omega)\right|\cos\Big(2\omega t + \angle H(j2\omega)\Big) \quad (15)$$

Hence, the amplitude component can be extracted by substituting the magnitude equation:

$$A_{\text{ripple}} = \frac{4}{3\pi}\,\frac{1}{\sqrt{1+(2\omega RC)^2}} \quad (16)$$

The ripple amplitude can be further simplified for beyond the cutoff, where $2\omega RC$ dominates and 1 becomes negligible:

$$A_{\text{ripple}} \approx \frac{2}{3\pi\,\omega RC} \quad (17)$$

The final formulas for the metrics become:

$$\text{Responsivity}(\rho) = \frac{\omega RC}{2\pi}\ln\left(\frac{1}{\varepsilon}\right) \quad (18)$$

$$\text{Stability}(\kappa) = \frac{2}{3\pi\omega RC} \quad (19)$$

valid for $f \geq f_c$, where $f_c = \dfrac{1}{2\pi RC}$ is the cutoff frequency.

$$\rho \times \kappa = \frac{1}{3\pi^2}\ln\left(\tfrac{1}{\varepsilon}\right) \quad (20)$$

This shows that Responsivity and Stability are intrinsically coupled through a fixed constant. Hence, these metrics are ideal performance descriptors, as they directly relate to each other, and their relationship is independent of frequency.

Utilizing the frequency range constraints provided, the following inequalities are obtained:

$$\rho \geq \frac{1}{2\pi}\ln\left(\frac{1}{\varepsilon}\right) \qquad \kappa \leq \frac{2}{3\pi} \approx 0.212 \quad (21)$$

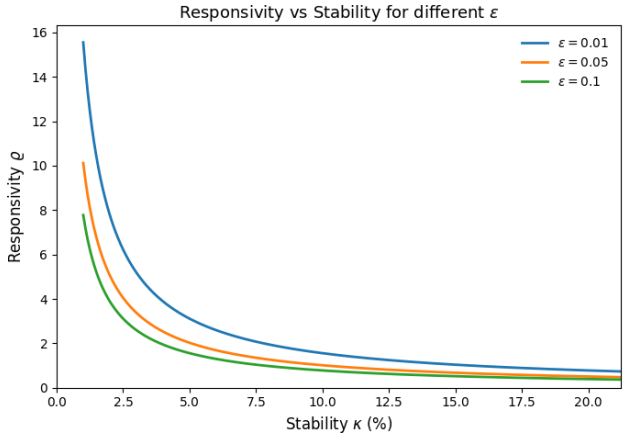

Figure 6: Relationship between Responsivity and Stability for different tolerance values.

VI. Standard Neuron Configuration and Training

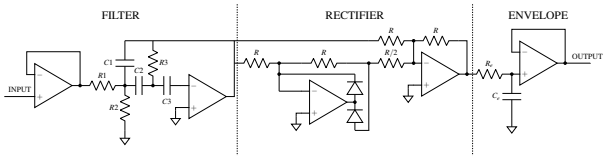

Figure 7: Full-wave rectifier-based envelope extraction circuit.

This circuit shows the standard neuron schematic. This circuit can be simulated and represented in LTspice; however, expressing its behavior as a comprehensive formula or system of equations better serves its purpose.

$$H_{BP}(s) = \frac{\frac{\omega_0}{Q}s}{s^2 + \frac{\omega_0}{Q}s + \omega_0^2} \quad (22)$$

This provides the following differential equation, which also incorporates the transient response:

$$\frac{d^2v_o}{dt^2} + \frac{\omega_0}{Q}\frac{dv_o}{dt} + \omega_0^2 v_o = \frac{\omega_0}{Q}\frac{dv_i}{dt} \quad (23)$$

The second equation is formulated using the RC differential equation and a square root approximation of the absolute value function to ensure differentiability for training purposes:

$$RC\frac{dv_e}{dt} + v_e = v_{in} \quad (24)$$

$$\frac{2}{3\pi\omega_0\kappa}\frac{dv_e}{dt} + v_e = \sqrt{(v_o)^2 + \varepsilon} \tag{25}$$

where ε is a small positive constant for numerical stability (e.g., $\varepsilon = 10^{-6}$) and RC is expressed in terms of stability(κ).

This system of equations encompasses the entire circuit whilst introducing differentiability:

$$\frac{d^2v_o}{dt^2} + \frac{\omega_0}{Q}\frac{dv_o}{dt} + \omega_0^2 v_o = \frac{\omega_0}{Q}\frac{dv_i}{dt} \tag{26}$$

$$\frac{2}{3\pi\omega_0\kappa}\frac{dv_e}{dt} + v_e = \sqrt{(v_o)^2 + \varepsilon_1} \tag{27}$$

$$\rho = \frac{\omega RC}{2\pi}\ln\left(\frac{1}{\varepsilon_2}\right) \qquad \kappa = \frac{2}{3\pi\omega RC} \tag{28}$$

$$\rho \geq \frac{1}{2\pi}\ln\left(\frac{1}{\varepsilon_2}\right) \qquad \kappa \leq \frac{2}{3\pi} \approx 0.212 \tag{29}$$

Where, v_i is the input signal, v_e is the output signal/envelope, ε_1 is the constant of the square root approximation, ρ is the responsiveness, ε_2 is the tolerance for ρ, and κ is the stability.

VII. Higher Order Configurations

The differential equations for higher-order resonant filters can be derived by raising the standard second-order bandpass transfer function to the power N:

$$H_{BP}(s) = \left(\frac{\frac{\omega_0}{Q}s}{s^2 + \frac{\omega_0}{Q}s + \omega_0^2}\right)^N \tag{30}$$

where N is the number of cascaded sections and the resulting filter is of order $2N$.

VIII. Pure AC Configuration

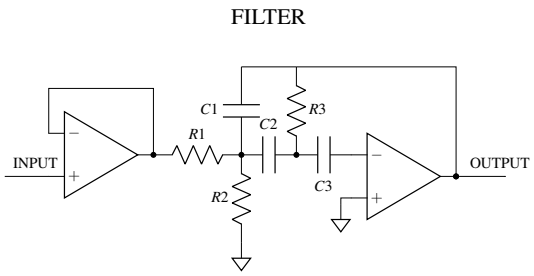

Figure 8: Full-wave rectifier-based envelope extraction circuit.

A pure AC alternative may also be used, depending on the application. This would have the following equation only:

$$\frac{d^2v_o}{dt^2} + \frac{\omega_0}{Q}\frac{dv_o}{dt} + \omega_0^2 v_o = \frac{\omega_0}{Q}\frac{dv_i}{dt} \tag{31}$$

IX. Data Generation

To evaluate the performance of the proposed Trainable Resonant Neuron (TRN) architecture against a Fixed Filter Bank (FFB) baseline, a controlled, synthetic dataset of multi-tone audio signals was constructed:

$$x^+(t) = \sum_{i=1}^{N} A_i\big(1+\varepsilon_{A_i}\big)\,\sin\Big(2\pi f_i\big(1+\varepsilon_{f_i}\big)t + \phi_i\Big) \tag{32}$$

where $x^+(t)$ is the input positive signal, N is the number of sinusoidal components, A_i and f_i are the nominal amplitude and frequency of the i-th sinusoid, ϕ_i is its phase, and t is time. The perturbation factors ε_{A_i} and ε_{f_i} are drawn from uniform distributions $\mathscr{U}(-0.2, 0.2)$ and $\mathscr{U}(-0.1, 0.1)$, corresponding to the amplitude and frequency variations of $\pm 20\%$ and $\pm 10\%$, respectively.

For each positive signal, a negative example $x^-(t)$ is generated by corrupting a random subset of its sinusoids, replacing them with new components of random frequency, phase, and amplitude, yielding negatives that range from partially similar, to entirely different waveforms.

X. Computational Efficiency

For computational efficiency due to hardware constraints, only the steady-state solution of the bandpass differential equation will be utilized:

$$|H_{BP}(j\omega)| = \frac{\frac{\omega_0}{Q}\omega}{\sqrt{(\omega_0^2 - \omega^2)^2 + \left(\frac{\omega_0\omega}{Q}\right)^2}} \tag{33}$$

$$\angle H_{BP}(j\omega) = \arctan\left(\frac{\frac{\omega_0\omega}{Q}}{\omega_0^2 - \omega^2}\right) \tag{34}$$

$$v_o(t) = \sum_{i=1}^{N} A_i\,|H_{BP}(j\omega_i)|\,\sin\Big(\omega_i t + \phi_i + \angle H_{BP}(j\omega_i)\Big) \tag{35}$$

Where, $\omega_i = 2\pi f_i$ is the angular frequency of the i-th sinusoid.

Furthermore, for computational efficiency, the training and testing tones will be limited to a frequency range of 500–5,000 Hz.

Due to the computational cost associated with evaluating the differential equation

$$\frac{2}{3\pi\omega_0\kappa}\frac{dv_e}{dt} + v_e = \sqrt{(v_o)^2 + \varepsilon_1} \tag{36}$$

The system can by approximated by considering only the resultant DC component. This can be efficiently obtained by taking the time-average of the rectified signal:

$$v_e \approx \frac{1}{T}\int_0^T \sqrt{v_o(t)^2 + \varepsilon_1}\, dt \tag{37}$$

This approach condenses the entire signal into a single scalar per TRN, allowing the subsequent neural network to process just this value rather than the full time series. For discrete-time signals with N samples, this becomes the corresponding summation:

$$v_e \approx \frac{1}{N}\sum_{n=0}^{N-1} \sqrt{v_o[n]^2 + \varepsilon_1} \tag{38}$$

XI. Experimental Setup

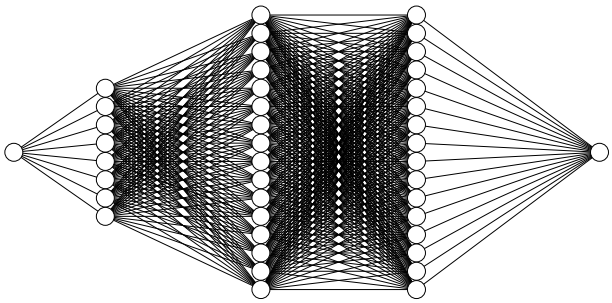

Figure 9: Neural network comprised of a single input source, a Trainable Resonant Neuron (TRN) layer, two 16-unit hidden layers, and a sigmoid output layer.

The neural network employs a clipped Rectified Linear Unit (ReLU) activation between 0 and 2.0 for analog circuit realism, implemented using a hard tan hyperbolic (hardtanh) function for improved training.

Center frequencies f_0 were chosen as the interior N points of a logarithmic frequency partition between 500–5000 Hz, such that the -3 dB points of adjacent filters intersect. For the edge filters, the -3 dB points are set at the boundary frequencies.

$$f_{i,-3\text{dB}} = f_{\min}\left(\frac{f_{\max}}{f_{\min}}\right)^{\frac{i}{N}} \tag{39}$$

where $f_{i,-3\text{dB}}$ denotes the i-th -3 dB frequency point, $f_{\min}$ and $f_{\max}$ are the lower and upper frequency bounds, respectively, and N is the number of filters. The index i ranges from 0 to N, producing $N+1$ logarithmically spaced points inclusive of the boundary frequencies. These frequencies correspond to the -3 dB cutoff frequencies of the filter bank, with each pair $(f_{i,-3\text{dB}}, f_{i+1,-3\text{dB}})$ defining the lower and upper cutoff of the i-th filter. The center frequency lies at their logarithmic midpoint:

$$f_{0,i} = \sqrt{f_{i,-3\text{dB}} \cdot f_{i+1,-3\text{dB}}} = f_{\min}\left(\frac{f_{\max}}{f_{\min}}\right)^{\frac{2i+1}{2N}} \tag{40}$$

The quality factor for each filter can be found

$$Q = \frac{f_{0,i}}{f_{i+1,-3\text{dB}} - f_{i,-3\text{dB}}} \tag{41}$$

$$Q = \frac{1}{\left(\frac{f_{\max}}{f_{\min}}\right)^{1/(2N)} - \left(\frac{f_{\max}}{f_{\min}}\right)^{-1/(2N)}} \tag{42}$$

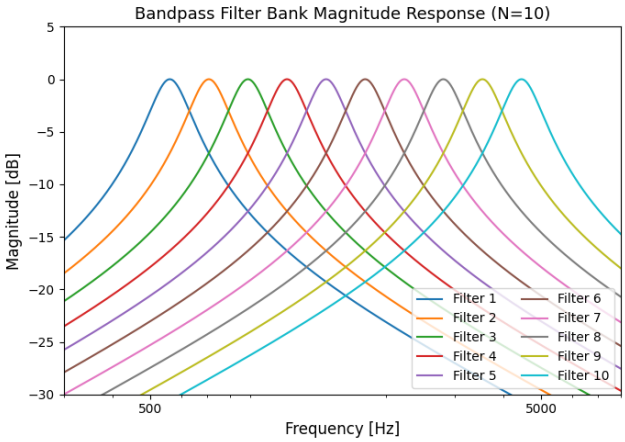

Figure 10: Bandpass filter bank example orientation for N=10.

The same spacing and quality factor are used as the initial guess for the parameters of the Trainable Resonant Neuron layer.

XII. Evaluation

The evaluation methodology involves running five independent repetitions of the entire experiment. In each run, the model is trained for five epochs on a balanced set of 10,000 samples (5,000 positive and 5,000 negative), and a batch size of 128. The model is then evaluated on a separate, held-out test set of 10,000 balanced samples. The evaluated key metrics are as follows:

I. *Accuracy*: The ratio of correct predictions (True Positives + True Negatives) to the total samples tested.

II. *Loss*: Measures the difference between predicted probabilities and true labels (Binary Cross-Entropy

Loss).

III. *AUC (Area Under ROC Curve)*: The model's ability to distinguish between classes across all possible thresholds, providing a threshold-independent measure of performance.

IV. *Precision*: The fraction of positive predictions that were correct (minimizes False Positives).

V. *Recall*: The fraction of actual positive cases that were correctly identified (minimizes False Negatives).

VI. *F1 Score*: The harmonic mean of Precision and Recall, providing a single score that balances both types of classification error.

The first set of evaluations used the following dataset:

$$\texttt{base\_freqs} = \begin{bmatrix} 500 & 800 & 1200 & 1800 & 2500 & 3200 \\ 3800 & 4200 & 4500 & 4700 & 4900 & 5000 \end{bmatrix}$$

$$\texttt{base\_amps} = \begin{bmatrix} 0.87 & 0.36 & 0.68 & 0.94 & 0.52 & 0.79 \\ 0.41 & 0.61 & 0.48 & 0.72 & 0.57 & 0.88 \end{bmatrix}$$

The frequencies are distributed roughly in an even manner, which is reflected in the evaluation results.

The second set adopts a more challenging configuration, incorporating white Gaussian noise ($\sigma = 2.0$) and a large, irregular frequency arrangement. Negative samples were generated in a power-invariant manner: for each component targeted for corruption, the amplitude was preserved while the frequency was shifted slightly beyond its nominal variability margin $\pm\mathscr{U}(0.1, 0.2)$. This prevents the network from exploiting simple power differences, enforcing a more rigorous test and reducing costly false positives.

$$\texttt{base\_freqs} = \begin{bmatrix} 515 & 595 & 680 & 805 & 950 \\ 1110 & 1260 & 1385 & 1490 & 3100 \\ 3600 & 5600 & 6050 & 6420 & 6980 \\ 7550 & 7880 & 8150 & 8490 & 12150 \\ 12800 & 13900 & 18200 & 19100 & 19990 \end{bmatrix}$$

$$\texttt{base\_amps} = \begin{bmatrix} 0.85 & 0.33 & 0.92 & 0.51 & 0.76 \\ 0.22 & 0.98 & 0.45 & 0.61 & 0.88 \\ 0.19 & 0.70 & 0.59 & 0.94 & 0.29 \\ 0.73 & 0.15 & 0.66 & 0.89 & 0.41 \\ 0.99 & 0.36 & 0.82 & 0.54 & 0.17 \end{bmatrix}$$

This approach is used instead of adjusting the efficiency metric by penalizing false positives according to the dissimilarity between negative and positive signals. Such schemes are inherently abstract, relying on post hoc weighting rather than shaping the test data itself. In contrast, the proposed method embeds this difficulty directly into signal generation, keeping negative samples power-invariant but subtly frequency-shifted, resulting in a more principled and concrete evaluation.

XIII. Conclusion

The results establish a clear relationship between architectural design, spectral complexity, and classification performance across two distinct tasks. Increasing the complexity parameter N consistently improved performance for both the Fixed Filter Bank (FFB) and the Trained Resonator Network (TRN) models, as reflected in key metrics such as Accuracy, AUC, and F1 Score. This trend highlights the central role of higher spectral resolution in enabling more effective feature extraction for classification tasks in this domain.

A critical pattern emerges in the comparative analysis between the two architectures: the relative performance advantage of the TRN correlates strongly with the intrinsic complexity of the classification problem. For Dataset 1, which represents a simpler, linearly separable classification scenario, both architectures achieved highly competitive peak performance (approaching 95% Accuracy). This result indicates that static, hand-crafted spectral filters are sufficient when class-discriminative features are coarse and easily captured through fixed frequency channels.

In contrast, for Dataset 2—characterized by substantially higher spectral and temporal complexity—the performance gap between the architectures widened markedly. The FFB model achieved its maximum Accuracy of 78.45% at $N = 11$, while the TRN, benefiting from its trainable resonance mechanisms, reached 84.69% at the same complexity level. Although this absolute difference of approximately 6% may initially appear modest, it serves as compelling evidence of the TRN's capacity to exploit finer spectral structure that remains inaccessible to Fixed Filter Banks (FFB). Importantly, this gap should be interpreted not as the upper bound of the TRN's advantage, but rather as a lower-bound estimate obtained under controlled test conditions.

The adaptive nature of the TRN implies that its relative performance margin is expected to increase as task complexity grows. In scenarios involving richer spectral manifolds, non-stationary class boundaries, or intricate nonlinear feature interactions, fixed spectral decompositions are fundamentally limited in their ability to capture discriminative information. In contrast, the TRN dynamically aligns its resonance characteristics to the data distribution, enabling more efficient representation of subtle, high-dimensional structure. Consequently, for more demanding real-world classification problems, the performance margin observed in this study is anticipated to expand significantly beyond the 6% observed here, positioning the TRN as the preferred architecture in domains where data complexity and model robustness are paramount.

XIV. Acknowledgment

This work was conducted independently and received no external funding or sponsorship.

XV. Dataset 1 Results

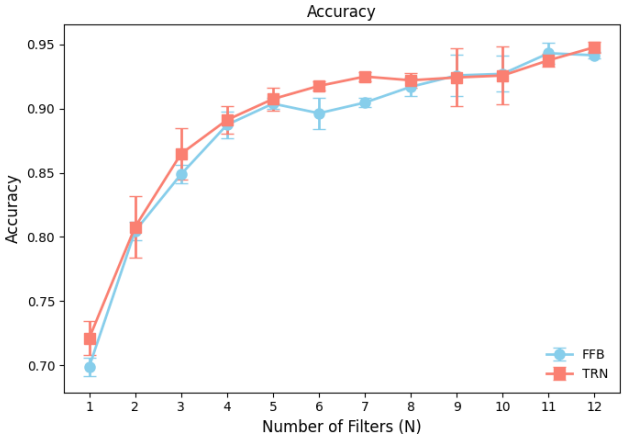

Figure 11: Accuracy comparison for TRN vs FFB.

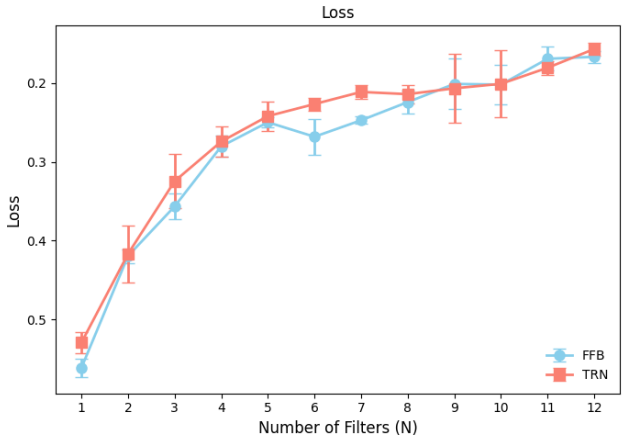

Figure 12: Loss comparison for TRN vs FFB.

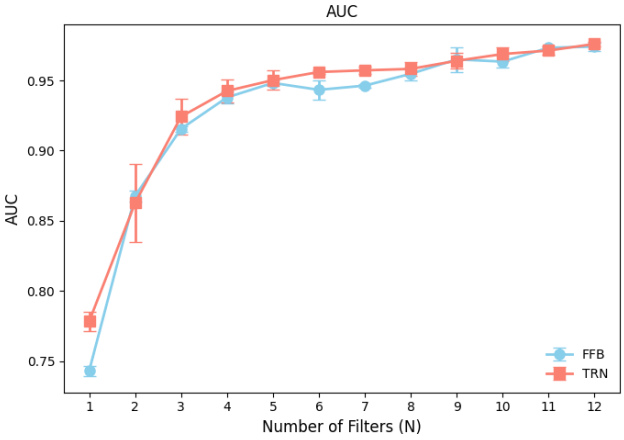

Figure 13: AUC comparison for TRN vs FFB.

XVI. Dataset 1 Results Continuation

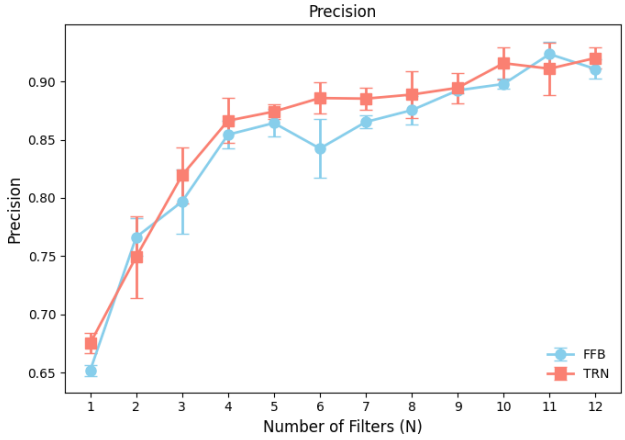

Figure 14: Precision comparison for TRN vs FFB.

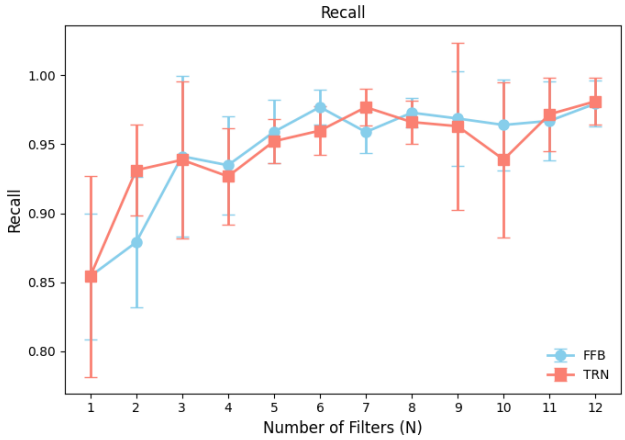

Figure 15: Recall comparison for TRN vs FFB.

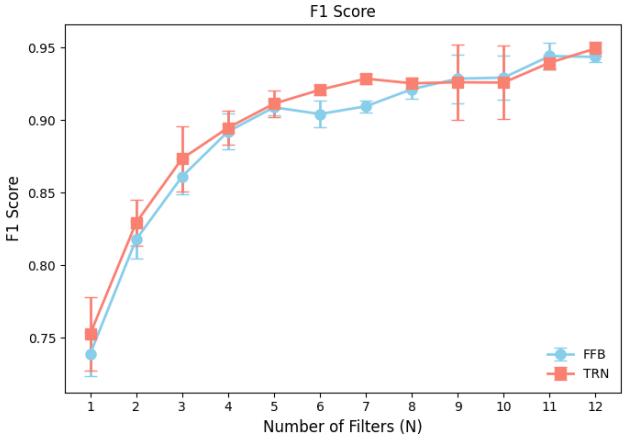

Figure 16: F1 score comparison for TRN vs FFB.

Table 2: Average Evaluation Metrics (Mean ± Std. Dev.) for Dataset 1.

N	Model	Accuracy	Loss	AUC	Precision	Recall	F1 Score
1	FFB	0.6988 ± 0.0070	0.5615 ± 0.0112	0.7431 ± 0.0035	0.6519 ± 0.0044	0.8542 ± 0.0456	0.7388 ± 0.0151
1	TRN	0.7210 ± 0.0132	0.5295 ± 0.0138	0.7783 ± 0.0070	0.6753 ± 0.0086	0.8541 ± 0.0727	0.7525 ± 0.0251
2	FFB	0.8046 ± 0.0070	0.4194 ± 0.0087	0.8677 ± 0.0040	0.7663 ± 0.0160	0.8789 ± 0.0473	0.8177 ± 0.0130
2	TRN	0.8078 ± 0.0243	0.4168 ± 0.0360	0.8627 ± 0.0280	0.7494 ± 0.0350	0.9312 ± 0.0331	0.8293 ± 0.0161
3	FFB	0.8488 ± 0.0073	0.3564 ± 0.0162	0.9155 ± 0.0019	0.7969 ± 0.0278	0.9413 ± 0.0584	0.8611 ± 0.0122
3	TRN	0.8648 ± 0.0201	0.3244 ± 0.0340	0.9242 ± 0.0129	0.8194 ± 0.0243	0.9387 ± 0.0572	0.8735 ± 0.0224
4	FFB	0.8874 ± 0.0103	0.2802 ± 0.0137	0.9379 ± 0.0041	0.8543 ± 0.0119	0.9348 ± 0.0355	0.8922 ± 0.0123
4	TRN	0.8912 ± 0.0105	0.2741 ± 0.0193	0.9425 ± 0.0079	0.8663 ± 0.0194	0.9267 ± 0.0352	0.8948 ± 0.0116
5	FFB	0.9039 ± 0.0042	0.2496 ± 0.0063	0.9482 ± 0.0018	0.8643 ± 0.0119	0.9590 ± 0.0230	0.9089 ± 0.0053
5	TRN	0.9075 ± 0.0090	0.2418 ± 0.0186	0.9502 ± 0.0070	0.8741 ± 0.0066	0.9522 ± 0.0160	0.9114 ± 0.0091
6	FFB	0.8964 ± 0.0121	0.2680 ± 0.0230	0.9433 ± 0.0068	0.8424 ± 0.0254	0.9770 ± 0.0128	0.9043 ± 0.0092
6	TRN	0.9177 ± 0.0036	0.2265 ± 0.0080	0.9560 ± 0.0014	0.8857 ± 0.0135	0.9599 ± 0.0176	0.9210 ± 0.0035
7	FFB	0.9047 ± 0.0034	0.2471 ± 0.0047	0.9463 ± 0.0013	0.8652 ± 0.0055	0.9589 ± 0.0150	0.9096 ± 0.0042
7	TRN	0.9250 ± 0.0036	0.2111 ± 0.0084	0.9572 ± 0.0026	0.8852 ± 0.0092	0.9769 ± 0.0131	0.9287 ± 0.0035
8	FFB	0.9170 ± 0.0072	0.2240 ± 0.0142	0.9547 ± 0.0046	0.8753 ± 0.0121	0.9729 ± 0.0104	0.9214 ± 0.0065
8	TRN	0.9221 ± 0.0056	0.2139 ± 0.0117	0.9582 ± 0.0047	0.8887 ± 0.0201	0.9661 ± 0.0157	0.9255 ± 0.0040
9	FFB	0.9259 ± 0.0158	0.2009 ± 0.0319	0.9650 ± 0.0088	0.8923 ± 0.0023	0.9687 ± 0.0344	0.9287 ± 0.0168
9	TRN	0.9243 ± 0.0226	0.2065 ± 0.0438	0.9640 ± 0.0056	0.8945 ± 0.0130	0.9631 ± 0.0608	0.9262 ± 0.0258
10	FFB	0.9271 ± 0.0140	0.2018 ± 0.0253	0.9634 ± 0.0045	0.8978 ± 0.0043	0.9640 ± 0.0332	0.9294 ± 0.0152
10	TRN	0.9258 ± 0.0224	0.2008 ± 0.0427	0.9688 ± 0.0048	0.9156 ± 0.0136	0.9389 ± 0.0563	0.9260 ± 0.0254
11	FFB	0.9432 ± 0.0081	0.1690 ± 0.0151	0.9733 ± 0.0007	0.9234 ± 0.0109	0.9670 ± 0.0284	0.9443 ± 0.0092
11	TRN	0.9377 ± 0.0051	0.1804 ± 0.0088	0.9713 ± 0.0029	0.9109 ± 0.0223	0.9718 ± 0.0267	0.9397 ± 0.0049
12	FFB	0.9416 ± 0.0027	0.1665 ± 0.0076	0.9741 ± 0.0030	0.9106 ± 0.0085	0.9795 ± 0.0166	0.9437 ± 0.0034
12	TRN	0.9479 ± 0.0045	0.1566 ± 0.0084	0.9760 ± 0.0018	0.9201 ± 0.0092	0.9812 ± 0.0172	0.9495 ± 0.0049

XVII. Dataset 2 Results

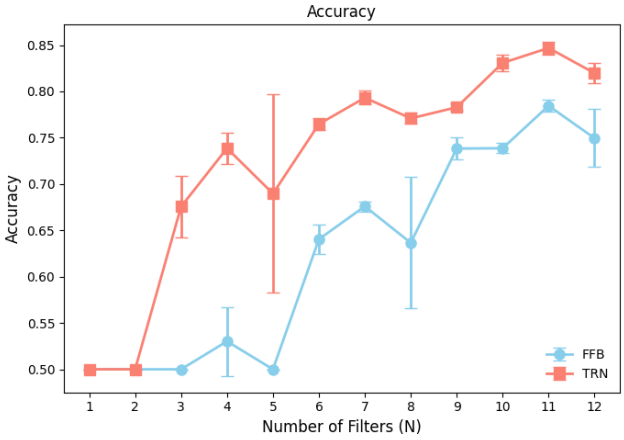

Figure 17: Accuracy comparison for TRN vs FFB.

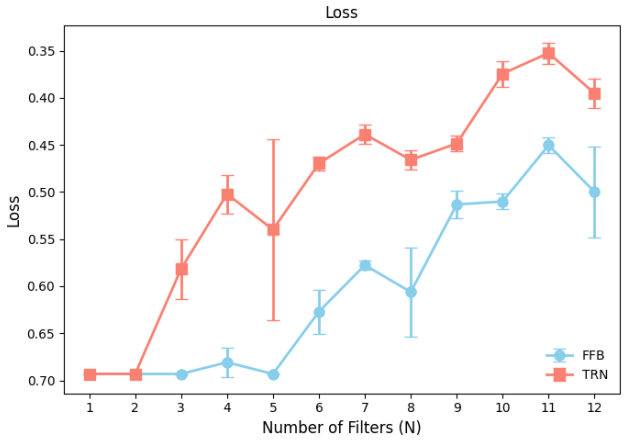

Figure 18: Loss comparison for TRN vs FFB.

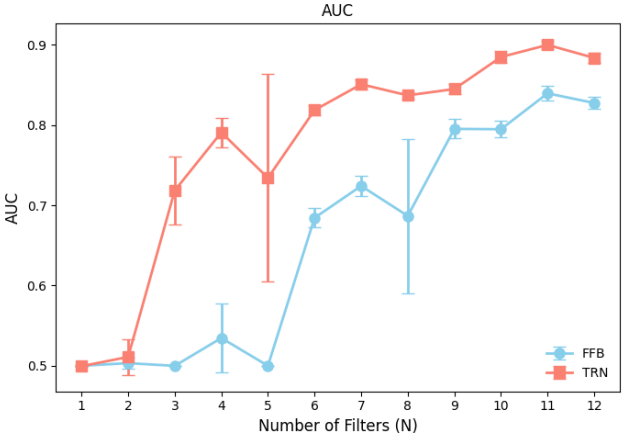

Figure 19: AUC comparison for TRN vs FFB.

XVIII. Dataset 2 Results Continuation

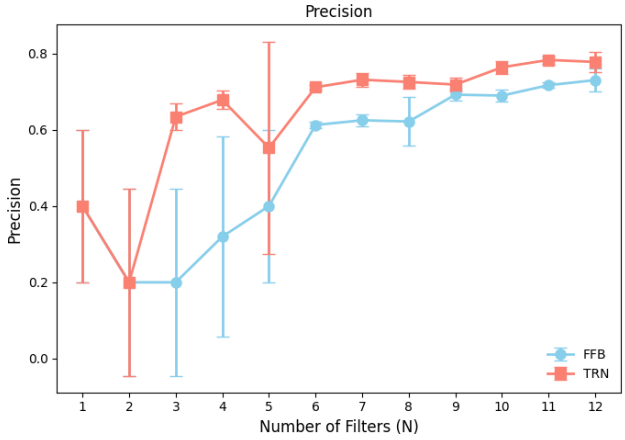

Figure 20: Precision comparison for TRN vs FFB.

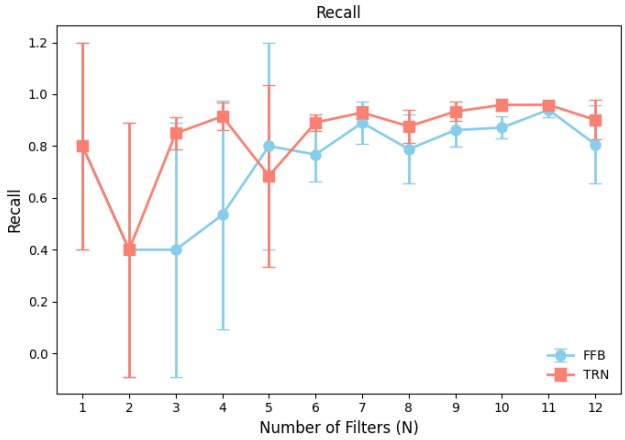

Figure 21: Recall comparison for TRN vs FFB.

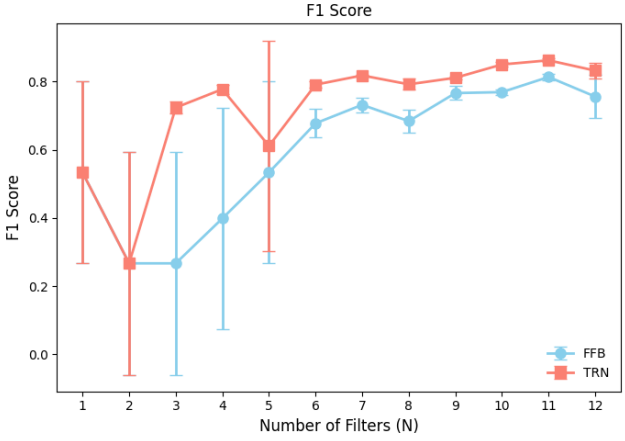

Figure 22: F1 score comparison for TRN vs FFB.

Table 3: Average Evaluation Metrics (Mean ± Std. Dev.) for Dataset 2.

N	Model	Accuracy	Loss	AUC	Precision	Recall	F1 Score
1	FFB	0.5000 ± 0.0000	0.6932 ± 0.0001	0.5000 ± 0.0001	0.4000 ± 0.2000	0.8000 ± 0.4000	0.5333 ± 0.2667
1	TRN	0.5000 ± 0.0000	0.6932 ± 0.0001	0.4995 ± 0.0010	0.4000 ± 0.2000	0.8000 ± 0.4000	0.5333 ± 0.2667
2	FFB	0.5000 ± 0.0000	0.6932 ± 0.0000	0.5033 ± 0.0071	0.2000 ± 0.2449	0.4000 ± 0.4899	0.2667 ± 0.3266
2	TRN	0.5000 ± 0.0000	0.6932 ± 0.0000	0.5110 ± 0.0221	0.2000 ± 0.2449	0.4000 ± 0.4899	0.2667 ± 0.3266
3	FFB	0.5000 ± 0.0000	0.6932 ± 0.0000	0.4999 ± 0.0001	0.2000 ± 0.2449	0.4000 ± 0.4899	0.2667 ± 0.3266
3	TRN	0.6756 ± 0.0329	0.5817 ± 0.0317	0.7185 ± 0.0426	0.6343 ± 0.0349	0.8493 ± 0.0609	0.7236 ± 0.0181
4	FFB	0.5302 ± 0.0371	0.6808 ± 0.0153	0.5346 ± 0.0425	0.3199 ± 0.2619	0.5345 ± 0.4418	0.3984 ± 0.3255
4	TRN	0.7386 ± 0.0172	0.5021 ± 0.0205	0.7909 ± 0.0181	0.6785 ± 0.0247	0.9148 ± 0.0548	0.7775 ± 0.0135
5	FFB	0.5000 ± 0.0000	0.6932 ± 0.0001	0.5000 ± 0.0000	0.4000 ± 0.2000	0.8000 ± 0.4000	0.5333 ± 0.2667
5	TRN	0.6897 ± 0.1069	0.5399 ± 0.0958	0.7342 ± 0.1292	0.5528 ± 0.2791	0.6847 ± 0.3496	0.6112 ± 0.3095
6	FFB	0.6404 ± 0.0161	0.6270 ± 0.0233	0.6844 ± 0.0119	0.6127 ± 0.0079	0.7672 ± 0.1052	0.6775 ± 0.0417
6	TRN	0.7647 ± 0.0063	0.4697 ± 0.0070	0.8187 ± 0.0063	0.7117 ± 0.0068	0.8909 ± 0.0324	0.7909 ± 0.0100
7	FFB	0.6760 ± 0.0055	0.5776 ± 0.0049	0.7242 ± 0.0128	0.6250 ± 0.0153	0.8901 ± 0.0828	0.7317 ± 0.0219
7	TRN	0.7932 ± 0.0074	0.4386 ± 0.0102	0.8509 ± 0.0063	0.7314 ± 0.0186	0.9294 ± 0.0260	0.8180 ± 0.0023
8	FFB	0.6365 ± 0.0709	0.6062 ± 0.0475	0.6868 ± 0.0963	0.6218 ± 0.0633	0.7880 ± 0.1327	0.6839 ± 0.0346
8	TRN	0.7709 ± 0.0059	0.4659 ± 0.0104	0.8371 ± 0.0044	0.7256 ± 0.0183	0.8761 ± 0.0640	0.7919 ± 0.0164
9	FFB	0.7383 ± 0.0118	0.5132 ± 0.0151	0.7953 ± 0.0121	0.6923 ± 0.0150	0.8617 ± 0.0628	0.7662 ± 0.0201
9	TRN	0.7829 ± 0.0038	0.4484 ± 0.0079	0.8450 ± 0.0065	0.7186 ± 0.0172	0.9332 ± 0.0377	0.8111 ± 0.0039
10	FFB	0.7387 ± 0.0056	0.5100 ± 0.0082	0.7949 ± 0.0101	0.6897 ± 0.0154	0.8711 ± 0.0427	0.7690 ± 0.0088
10	TRN	0.8308 ± 0.0091	0.3743 ± 0.0137	0.8849 ± 0.0077	0.7640 ± 0.0169	0.9590 ± 0.0233	0.8501 ± 0.0064
11	FFB	0.7845 ± 0.0064	0.4503 ± 0.0080	0.8396 ± 0.0090	0.7171 ± 0.0072	0.9403 ± 0.0293	0.8134 ± 0.0085
11	TRN	0.8469 ± 0.0071	0.3524 ± 0.0113	0.9000 ± 0.0056	0.7834 ± 0.0093	0.9591 ± 0.0106	0.8623 ± 0.0059
12	FFB	0.7495 ± 0.0313	0.4999 ± 0.0486	0.8276 ± 0.0071	0.7304 ± 0.0305	0.8065 ± 0.1505	0.7560 ± 0.0633
12	TRN	0.8197 ± 0.0111	0.3952 ± 0.0157	0.8838 ± 0.0062	0.7780 ± 0.0267	0.9012 ± 0.0762	0.8320 ± 0.0219

XIX. Acknowledgment

This work was conducted independently and received no external funding or sponsorship.

References

[1] A. Mucciardi, D. Cleveland, and E. Orr, "Trainable nonlinear filters," in *Proc. IEEE Int. Conf. Acoustics, Speech, and Signal Processing (ICASSP)*, Hartford, CT, USA, 1977, pp. 112–115. doi: 10.1109/ICASSP.1977.1170263.

[2] B. Zheng, S. Yuan, G. Slabaugh, and A. Leonardis, "Image demoireing with learnable bandpass filters," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, Seattle, WA, USA, 2020, pp. 3633–3642. doi: 10.1109/CVPR42600.2020.00369.

[3] H. P. Tukuljac, B. Ricaud, N. Aspert, and L. Colbois, "Learnable filter-banks for CNN-based audio applications," in *Proc. Northern Lights Deep Learning Workshop*, vol. 3, Mar. 2022. doi: 10.7557/18.6279.

[4] X. Cai and S. Ko, "Development of parametric filter banks for sound feature extraction," *IEEE Access*, vol. 11, pp. 109856–109867, 2023. doi: 10.1109/ACCESS.2023.3321798.

[5] M. Ravanelli and Y. Bengio, "Speaker recognition from raw waveform with SincNet," in *Proc. IEEE Spoken Language Technology Workshop (SLT)*, Athens, Greece, 2018, pp. 1021–1028. doi: 10.1109/SLT.2018.8639585.

[6] N. Strisciuglio, M. Vento, and N. Petkov, "Learning representations of sound using trainable COPE feature extractors," *Pattern Recognition*, vol. 92, pp. 25–36, 2019. doi: 10.1016/j.patcog.2019.03.016.

[7] S. Ludwig, S. Bakas, D. A. Adamos, N. Laskaris, Y. Panagakis, and S. Zafeiriou, "EEGminer: discovering interpretable features of brain activity with learnable filters," *J. Neural Eng.*, vol. 21, no. 3, p. 036010, May 2024. doi: 10.1088/1741-2552/ad44d7.

[8] K. Imamura, T. Nakamura, K. Yatabe, and H. Saruwatari, "Neural analog filter for sampling-frequency-independent convolutional layer," *APSIPA Trans. Signal Inf. Process.*, vol. 13, no. 1, p. e28, 2024. doi: 10.1561/116.20230082.

[9] J. Hu, W. Goh, and Y. Gao, "AI-powered agile analog circuit design and optimization," arXiv preprint arXiv:2505.03750, Apr. 2025. doi: 10.48550/arXiv.2505.03750.

Appendix

The only notable and non-obvious step in the derivation concerns the transition from Equation 4 to Equation 5, Equation 22 to Equation 23, and Equation 41 to Equation 42.

Equation 4 to Equation 5 derivation:

$$|H_{BP}(j\omega)|^N = \left(\frac{\frac{\omega_0}{Q}\,\omega}{\sqrt{(\omega_0^2-\omega^2)^2+\left(\frac{\omega_0\omega}{Q}\right)^2}} \right)^N$$

$$|H_{BP}(j\omega_{\text{-3dB}})|^N = \frac{1}{\sqrt{2}}|H_{BP}(\omega_0)|^N$$

$$\left(\frac{\left(\frac{\omega_0}{Q}\,\omega_{\text{-3dB}}\right)^2}{(\omega_0^2-\omega_{\text{-3dB}}^2)^2+\left(\frac{\omega_0\omega_{\text{-3dB}}}{Q}\right)^2} \right)^N = \frac{1}{2}$$

$$\frac{\left(\frac{\omega_0}{Q}\,\omega_{\text{-3dB}}\right)^2}{(\omega_0^2-\omega_{\text{-3dB}}^2)^2+\left(\frac{\omega_0\omega_{\text{-3dB}}}{Q}\right)^2} = 2^{-1/N}$$

$$(\omega_0^2-\omega_{\text{-3dB}}^2)^2+\left(\frac{\omega_0\omega_{\text{-3dB}}}{Q}\right)^2 = 2^{1/N}\left(\frac{\omega_0\omega_{\text{-3dB}}}{Q}\right)^2$$

$$(\omega_0^2-\omega_{\text{-3dB}}^2)^2 = (2^{1/N}-1)\left(\frac{\omega_0\omega_{\text{-3dB}}}{Q}\right)^2$$

$$\left(\frac{\omega_0^2-\omega_{\text{-3dB}}^2}{\omega_0\omega_{\text{-3dB}}}\right)^2 = \frac{2^{1/N}-1}{Q^2}$$

$$\left(\frac{\omega_0}{\omega_{\text{-3dB}}}-\frac{\omega_{\text{-3dB}}}{\omega_0}\right)^2 = \frac{2^{1/N}-1}{Q^2}$$

$$\frac{\omega_0}{\omega_{\text{-3dB}}}-\frac{\omega_{\text{-3dB}}}{\omega_0} = \frac{\sqrt{2^{1/N}-1}}{Q}$$

Let ω_1 and ω_2 be the lower and upper $-3\,\text{dB}$ frequencies respectively, with $\omega_1 < \omega_0 < \omega_2$. Then:

$$\frac{\omega_0}{\omega_1}-\frac{\omega_1}{\omega_0} = \frac{\sqrt{2^{1/N}-1}}{Q}$$

$$\frac{\omega_2}{\omega_0}-\frac{\omega_0}{\omega_2} = \frac{\sqrt{2^{1/N}-1}}{Q}$$

Multiply both equations by ω_0:

$$\frac{\omega_0^2}{\omega_1}-\omega_1 = \frac{\omega_0\sqrt{2^{1/N}-1}}{Q}$$

$$\omega_2-\frac{\omega_0^2}{\omega_2} = \frac{\omega_0\sqrt{2^{1/N}-1}}{Q}$$

Using the geometric mean property $\omega_1\omega_2 = \omega_0^2$:

$$\frac{\omega_0^2}{\omega_1} = \omega_2 \quad \text{and} \quad \frac{\omega_0^2}{\omega_2} = \omega_1$$

Substitute into the first multiplied equation:

$$\omega_2-\omega_1 = \frac{\omega_0\sqrt{2^{1/N}-1}}{Q}$$

Substitute into the second multiplied equation:

$$\omega_2-\omega_1 = \frac{\omega_0\sqrt{2^{1/N}-1}}{Q}$$

Thus, the bandwidth is:

$$\Delta\omega = \omega_2-\omega_1 = \frac{\omega_0}{Q}\sqrt{2^{1/N}-1}$$

The effective quality factor is:

$$Q_{\text{eff}}^{(N)} = \frac{\omega_0}{\Delta\omega} = \frac{\omega_0}{\frac{\omega_0}{Q}\sqrt{2^{1/N}-1}} = \frac{Q}{\sqrt{2^{1/N}-1}}$$

$$\boxed{Q_{\text{eff}}^{(N)} = \frac{Q}{\sqrt{2^{1/N}-1}}}$$

Equation 22 to Equation 23 derivation:

$$H_{BP}(s) = \frac{\frac{\omega_0}{Q}s}{s^2 + \frac{\omega_0}{Q}s + \omega_0^2}$$

Cross-multiplying in the Laplace domain:

$$V_o(s)\left(s^2 + \frac{\omega_0}{Q}s + \omega_0^2\right) = \frac{\omega_0}{Q}sV_i(s)$$

Expanding the left side:

$$s^2V_o(s) + \frac{\omega_0}{Q}sV_o(s) + \omega_0^2V_o(s) = \frac{\omega_0}{Q}sV_i(s)$$

Taking the inverse Laplace transform term-by-term:

$$\mathscr{L}^{-1}\{s^2V_o(s)\} = \frac{d^2v_o}{dt^2}$$

$$\mathscr{L}^{-1}\left\{\frac{\omega_0}{Q}sV_o(s)\right\} = \frac{\omega_0}{Q}\frac{dv_o}{dt}$$

$$\mathscr{L}^{-1}\{\omega_0^2V_o(s)\} = \omega_0^2v_o$$

$$\mathscr{L}^{-1}\left\{\frac{\omega_0}{Q}sV_i(s)\right\} = \frac{\omega_0}{Q}\frac{dv_i}{dt}$$

Thus, the final time-domain differential equation is obtained:

$$\boxed{\frac{d^2v_o}{dt^2} + \frac{\omega_0}{Q}\frac{dv_o}{dt} + \omega_0^2v_o = \frac{\omega_0}{Q}\frac{dv_i}{dt}}$$

Equation 41 to Equation 42 derivation:

$$Q_i = \frac{f_{0,i}}{f_{i+1,-3\text{dB}} - f_{i,-3\text{dB}}}$$

The center frequency is given as:

$$f_{0,i} = \sqrt{f_{i,-3\text{dB}}\, f_{i+1,-3\text{dB}}}$$

This can be substituted back into the equation

$$Q_i = \frac{\sqrt{f_{i,-3\text{dB}}\, f_{i+1,-3\text{dB}}}}{f_{i+1,-3\text{dB}} - f_{i,-3\text{dB}}}$$

Due to constant logarithmic spacing

$$f_{i+1,-3\text{dB}} = r\, f_{i,-3\text{dB}}$$

$$r = \frac{f_{i+1,-3\text{dB}}}{f_{i,-3\text{dB}}} = \left(\frac{f_{\max}}{f_{\min}}\right)^{1/N}$$

$$Q_i = \frac{\sqrt{r}\, f_{i,-3\text{dB}}}{(r-1)f_{i,-3\text{dB}}} = \frac{\sqrt{r}}{r-1}$$

$$Q = \frac{\left(\frac{f_{\max}}{f_{\min}}\right)^{1/(2N)}}{\left(\frac{f_{\max}}{f_{\min}}\right)^{1/N} - 1}$$

$$Q = \frac{\left(\frac{f_{\max}}{f_{\min}}\right)^{1/(2N)}\left(\frac{f_{\max}}{f_{\min}}\right)^{-1/(2N)}}{\left(\frac{f_{\max}}{f_{\min}}\right)^{1/N}\left(\frac{f_{\max}}{f_{\min}}\right)^{-1/(2N)} - \left(\frac{f_{\max}}{f_{\min}}\right)^{-1/(2N)}}$$

Thus, the final simplified equation is obtained:

$$\boxed{Q = \frac{1}{\left(\frac{f_{\max}}{f_{\min}}\right)^{1/(2N)} - \left(\frac{f_{\max}}{f_{\min}}\right)^{-1/(2N)}}}$$